Palacan

TECHNICAL DOCUMENT

Architecture and Threat Model

What runs where, what the server can and cannot see, which attacks Palacan resists, and its declared limits.

This document is generated from the public repository, where the most recent version is always available: github.com/Sbirringa/palacan-client

Version of 9 November 2026, from commit `d1261f8`

Source: `docs/ARCHITECTURE.md`

© 2026 Palacan. All rights reserved.

Palacan architecture

This document explains where each part of Palacan runs, what each part can see, and which attacks Palacan is designed to resist. The cryptographic details are in `PROTOCOL.md`; how to check all of this yourself is in `VERIFICATION.md`.

1. Components

Component	Where it runs	In this repository
Web app (React, Vite)	in the user's browser	yes: `frontend/my-react-app/vite-project/src`
Cryptography module (Rust: vodozemac, ML-KEM-768, ML-DSA-65)	in the browser, as WebAssembly	yes: `crypto/palacan-crypto`
Static website hosting (nginx, strict Content-Security-Policy)	server	configuration only: `frontend/my-react-app/vite-project/nginx`, `frontend/my-react-app/vite-project/sicurezza`
Backend API and WebSocket relay (Python, FastAPI, PostgreSQL)	server	no
TURN relay for calls	server	no
Palacan Verifica browser extension	in the user's browser	yes: `frontend/my-react-app/vite-project/estensione`

Why the server code is not published. All encryption and decryption happen on the user's device, in the code of this repository, before anything leaves it. The server only stores and forwards data that is already encrypted and never has the keys. To check that messages are protected, what matters is the code running on the device, and that code is here, together with a way to prove that the website serves exactly this code (`VERIFICATION.md`).

2. What runs where

On the device:

- generating and storing all private keys (Olm, Megolm, ML-KEM, ML-DSA, principal key, envelope keys);

- encrypting and decrypting messages, reactions, call signalling, group names and room keys;
- verifying every signature (device keys, prekeys, principal keys, envelope keys, release signature of the website);
- the encrypted local history and the encrypted key backup.

On the server:

- accounts (username and password hash; no email or phone number is required), sessions and rate limits;
- public keys, signatures and the encrypted backup, stored and handed out without being able to forge them;
- relaying encrypted messages and keys between devices, and storing encrypted messages until they are delivered or expire;
- for companies: separate workspaces, admin roles, employee accounts and tasks.

---

3. What the server can see

The server **cannot** read messages, reactions, call content, private group names or room keys, and it cannot forge signatures. Some metadata is unavoidable for delivering messages. Palacan reduces it as follows:

- **Senders.** For private chats the server does not store who sent a message. Room keys are delivered in sealed boxes: the server sees the recipient, not the sender (`PROTOCOL.md`, section 6). Inside a company workspace the sender is not stored either, and the admin cannot look it up.
- **Times.** For private chats, stored times are rounded to the hour.
- **Length.** For private chats, messages are padded, so their exact length is hidden.
- **Group names.** Private group names are end-to-end encrypted, also in invitations.
- **Retention.** Delivery data, expired items and dead sessions are cleaned up automatically. Messages can delete themselves after a time the user chooses. Company messages are never deleted automatically.

What the server **can** still see:

- which accounts are members of which chats, and when devices are connected (transport-level IP addresses and timing of connections), while it is delivering messages;

- that a call is taking place between members of a chat, and its start and end. The TURN relay sees the IP addresses of the two sides (each side's address is hidden from the other, not from the relay);
- public keys and signatures, the number of prekeys left, and the size and time of backup uploads.

---

4. Threat model

Palacan is designed to protect against:

- a **network attacker** who records or modifies traffic (TLS, plus end-to-end encryption);
- a **malicious or compromised server**, including one that modifies the stored data: it cannot read content, cannot replace keys without the change being detected (signatures, principal key, safety numbers), cannot remove the post-quantum protection of room keys (no downgrade), and cannot serve modified website code to users of the Palacan Verifica extension without being detected;
- **"harvest now, decrypt later"**: an attacker who records traffic today and has a quantum computer in the future (hybrid X25519 + ML-KEM-768 for room keys and sealed boxes, hybrid Ed25519 + ML-DSA-65 signatures);
- the **theft of a device's state**, with limits: past messages stay protected (forward secrecy of the ratchets), and security comes back after key rotation (Megolm sessions, Olm ratchet, post-quantum prekey renewal);
- an attacker who reads the **browser's files** (non-extractable device key, encrypted local storage).

Out of scope or limited (also listed in `formale/README.md`):

- **Trust on first use.** Keys are learned the first time they are seen. Users who want certainty compare the safety number or scan the QR code.
- **The website is delivered by the server.** Users who do not use the Palacan Verifica extension cannot check the code before it runs; a change targeted at one person can only be detected afterwards. An installable app signed with the release key is planned.
- **The release key** is held by one person, encrypted with a passphrase. Whoever obtained both the key file and the passphrase could sign a release.
- **Malware on an unlocked device**, physical access to an unlocked device, and attacks on the user's operating system or browser.

- **Denial of service.** A malicious server can refuse to deliver messages, keys or calls; it cannot read or forge them.
- **Metadata** listed in section 3.

---

5. Formal and automated checks

Every change to the code runs automatically:

- unit tests of the web app (Vitest) and of the server (pytest), and end-to-end tests with real browsers (Playwright);
- the reproducible build and the publication of its fingerprints;
- ProVerif, Kani, proptest and mutation tests on the cryptography.

The full register of guarantees, with the commit and the proof for each one, is in `formale/README.md`. That register is written in Italian; this document and `PROTOCOL.md` summarise it in English.